Machine Learning & Neural Networks – CM3015

Evaluating k-Nearest Neighbour and Decision Trees on Heart Disease Prediction With Varying Model Complexity

Midterm Report

by Alex Shaw

Abstract

This project compares two supervised classification algorithms, k-Nearest Neighbours and Decision Trees, on a heart disease dataset from scikit-learn. The effect of model complexity on accuracy was examined by varying key hyper-parameters. The results show that both models perform best at low complexity, with kNN achieving the highest overall accuracy and lowest variance.

Introduction

Supervised classification is a fundamental area of machine learning in which models are trained on labelled or seen data to predict the class of unseen or test data. A common challenge in classification is selecting an appropriate level of model complexity, as overly simple models may underfit while overly complex models may overfit. This project investigates these issues by comparing k-Nearest Neighbours and Decision Trees on a medical classification task (diagnosing heart disease based on medical survey), with particular focus on how hyper-parameter tuning influences performance through the bias-variance trade-off.

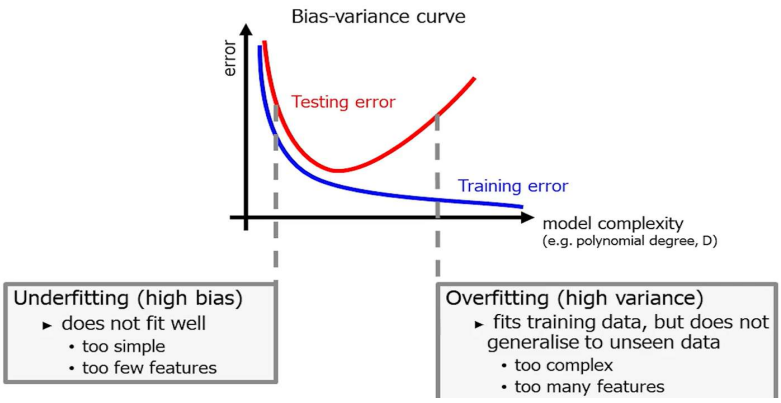
Background

The area of Machine Learning I have chosen to explore for this project is Supervised Classification. This is where machine learning algorithms are fed an input of labelled data (data which has been labelled for the purpose of for a particular task) in order to better predict where to sort or assign the unseen data it will be used on.

One of the main concerns when applying algorithms for classification is that the algorithm modelled on the training data could be either:

1) **Overfit** which is to say the algorithm has been made overly complex and tailored to the training data that, in addition to running more slowly, it will perform poorly on the unseen data as it has been made too specific when applied to seen data, and anything at all it is presented with outside of seen data, will be treated as alien and completely dissimilar to the training data in any respect. High variance and high degree polynomials are attributes of an overfit machine learning model. An example would be if the training data were colours being categorised, and it had learned to distinguish purple and red, well if it was presented with violet or scarlet it would read as non-associative in any way because of their small differences from the seen data.

2) **Underfit**, conversely, is to say that the model / algorithm has been made far too simple and not complex enough. It is beneficial in all algorithms to be as simple as possible to carry out a task however an algorithm which is underfit is one which, though simple, could be improved upon by adding complexity. It has been given a training set and loosely given parameters to distinguish which data belong and do not belong, meaning that it will likely be overly accepting of unseen data. If trained on a bowl of fruit, for example, instead of saying that a mandarin is more like an orange than a kiwi, it might instead simply say that they are all extremely similar simply on the basis of shape and the fact they are fruit, not taking into account taste, colour, texture or origin.



Seen above [1], these can be plotted on a Bias-Variance curve in order to see the sweet spot between a model which is overfit and one that is underfit. It shows that a model can be less than optimal for models on opposite sides of the complexity spectrum. The blue line above shows that, based on seen or training data, the error will decrease indefinitely as the complexity of the model increases. However, we can see from the red line that this is also true up to a point or “sweet spot”, after which it begins to climb again and you end up with an incredibly complex model with a high level of inaccuracy for the task at hand, and it is equally as ineffective as if the model were completely basic.

The two algorithms I have chosen to compare and evaluate for this project are **k-Nearest Neighbour (kNN)** and **Decision Trees.**

kNN

kNN is a classification method where new data is compared to previous examples seen by the training model. Rather than learning from a a set of training data, kNN instead only refers to the examples when it needs to, and makes determined predictions based on a specific set of parameters. It then calculated the “distance” or similarity between the new data being evaluated and the existing example it has seen, and usually using the formula for Euclidean distance, the k closest data points to the example are then selected and the most common of those are put forward as the designated prediction.

The value of k is the determining factor when considering how well kNN performs. A small value for k will make the algorithm overly sensitive to otherwise negligible noise, leading to overfitting. A value for k made to be too large could make patterns that should be highlighted slip under the radar and lead to underfitting. As kNN relies on Euclidean distance, it is also affected by the scale of the parameters chosen for evaluation. An income of tens of thousands could potentially be weighted higher than a boolean of whether a person is male or female, if the scale is not considered. This is why data scaling is an important consideration to make sure there is no data skewed by different metrics.

kNN is fast to set up but slower than a decision tree to determine and decide. As the algorithm simply stores the data, rather than training, it must evaluate each case beginning from zero. Also, referring to the Nearest Neighbour, each time a new piece of data is being evaluated, the model must consider all existing “neighbours” and consider their distance also, making it slower as the dataset being evaluated grows.

Decision Trees

A decision tree is a more intuitive evaluation model, essentially performing a series of binary calculations and whittling down the data until a conclusion of which class it belongs to is reached. Each of these binary decisions leading down and sprouting more

decisions, looks on paper like a tree, thus the name. Each decision has parameters, for example animal classification might look like “bigger or smaller than a car”, “4 or 2 legs”, “carnivore or herbivore”, and the data which does not meet the criteria is disregarded. The last nodes at the end of a path or branch are referred to as leaves following on the name *tree*.

One advantage of a decision tree is it's simplicity in theory. A series of binary questions to determine class is much easier to understand than other ML models. A disadvantage of a decision tree is that it can be overfit if it grows too large and too specific. The increased complexity can quickly become counterproductive. For example an extremely detailed and specific tree will perform excellently on the training data but poorly on unseen data as it will only be able to classify items identical to what was seen. Ensuring that the tree is not too specific will safeguard against this and avoid overfitting.

Contrary to kNN, decision trees are slow to set up and train, but much faster at making decisions and filtering data into the appropriate class. The lengthy process with building the algorithm behind the tree is primarily down to selecting the best node separation parameters and queries for the dataset, and seeing which works best. But when the tree is tested and built, it can separate and sort new data very quickly as it just follows one string of leaves down the tree.

While kNN and Decision Trees have different methodologies, they tackle the same problem which is to classify data and predict the best grouping of said data. I am interested to see how these different approaches, with their respective pros and cons, match up against each other on real data.

Other Algorithms Which Were Considered

In preparation for this project, I revisited other ML algorithms in the selection process. Linear regression was something I have studied in other modules as well as this one, getting into the maths behind it. However, I wanted to do my project specifically on classification algorithms and LR seemed less suited. LR is used to create a threshold for numerical values and predict that way what data will look like under certain circumstances. Price fluctuation or temperature would have been more suitable than categorising datasets.

Methodology

The aim of this project was to compare the performance of two supervised classification algorithms, k-Nearest Neighbours (kNN) and Decision Trees, on the same dataset. Once I had implemented these ML models, I decided to examine how changes in model complexity affect accuracy through the bias–variance comparison. To ensure a fair comparison, both algorithms were applied to the same dataset and evaluated using the same train–test split and performance metric.

The dataset used was the processed Cleveland Heart Disease dataset from the UCI Machine Learning Repository. This was ideal as it had already been cleaned and processed and the documentation was easy to read and understand even for a person with no medical training.

It consists of 303 patient records with 13 medical attributes and a value from 0 - 4 indicating the severity of heart disease, if any at all. Once I implemented a scaler, I realised that some data cells had missing data which was unsuitable for the scaler. So I returned to before the dataset was split and removed any cells with inappropriate data and then ran through the split and subsequent cells again.

Then the dataset was split into training and testing sets to evaluate how well each model generalised to unseen data. The training set was used to build the models, while the test set was held back and only used for final evaluation. This separation helps prevent overly optimistic results that can occur when a model is evaluated on data it has already seen.

The Decision Tree classifier was implemented using the scikit-learn library. Decision Trees do not rely on distance calculations, so feature scaling was not required. The complexity of the model was explored by altering the maximum depth of the tree, independent of other instances of the same model running. Shallow trees risk underfitting, while deeper trees can become overly complex and overfit the training data.

For k-Nearest Neighbours, the algorithm was implemented from scratch using standard Python and NumPy. This involved calculating Euclidean distance between patients based on their feature values, sorting neighbours by distance, and predicting class labels using a majority vote among the nearest neighbours. Implementing kNN manually was helpful to understand better how the Euclidean distance can be used in classification, rather than relying on library tools. I structured the implementation to follow the theoretical procedure of kNN, with distance calculated and predictions derived solely from designated neighbours (code is commented and explained in Jupyter notebook). Because kNN relies on distance calculations, feature scaling was applied to ensure that attributes with larger numerical ranges did not skew readings or results unfairly.

The bias–variance was explored with looped and automated hyper-parameter testing for both algorithms. For the Decision Tree, the maximum depth was varied across a range of values, and the accuracy was recorded for each depth. For kNN, the value of k was varied to test the quantity of nearest neighbours being taken into account, and accuracy was calculated for each value. These methodologies allowed complexity of both algorithms to be evaluated in a controlled environment.

Accuracy was chosen as the evaluation metric, as it provides a clear and intuitive measure of how often the predicted classifications matched the true labels. Accuracy was calculated by comparing predicted outcome for unseen patients with their heart disease diagnosis which had been hidden from the model until this point.

Results

The results of both models was evaluated based on accuracy of predictions on the test data once they had been trained appropriately. A correctly predicted diagnosis was sufficient to count toward a higher accuracy score. I also implemented loops in order to evaluate the accuracy of both models across a range of values for their respective complexity parameters.

For the Decision Tree classifier, accuracy was measured across a range of maximum tree depths. The model achieved its highest accuracy at a depth of 3, reaching approximately 63%. At very shallow depth (depth = 1), performance was lower, indicating underfitting. As tree depth increased beyond 3, accuracy steadily declined and eventually stabilised at a lower level, suggesting that increasing complexity did not improve generalisation performance.

For the k-Nearest Neighbours model, accuracy was evaluated for values of k ranging from 1 to 20. The highest accuracy was achieved at k = 2, with a peak accuracy of 70%. Accuracy was also relatively high for k = 1, though performance fluctuated more at very small values of k. As k increased, accuracy generally decreased and stabilised around 63–65%, indicating reduced sensitivity to local patterns at larger neighbourhood sizes.

Overall, kNN achieved a higher peak accuracy than the Decision Tree model as well

as having a smaller variation of accuracies. These results provided a basis for analysing underfitting, overfitting, and the bias–variance trade-off in the subsequent evaluation section.

Evaluation

I believe I have adequately demonstrate how model performance is strongly influenced by model complexity and the bias–variance curve. Both kNN and Decision Trees showed clear sensitivity to their respective complexity parameters, confirming that neither overly simple nor overly complex models generalise well to unseen data.

For the Decision Tree classifier, shallow trees performed best, with accuracy peaking at a depth of 3. Increasing the depth beyond this point drastically reduced test accuracy, indicating that the model had been overfit. While decision trees are capable of learning complex decision boundaries, it implies that excessive depth causes the model to memorise patterns specific to the training data rather than learning generalisable rules, and subsequently performing poorly on the unseen data and predictions.

The kNN model achieved its highest accuracy at very small values of k, particularly at k = 2. This indicates that the dataset also benefits from avoidance of an overly complex model, where predictions are based on close similarity between patients rather than global decision rules. As complexity value (k) increased, accuracy declined and before tapering off at a lower level, showing increased bias due to excessive smoothing. While kNN was more sensitive and fluctuated greatly with adjustments to its hyper-parameter, it outperformed Decision Tree model, suggesting that similarities between nearby data points in the dataset are especially important for making accurate classifications in this task.

A key limitation of this study is the use of a single train-test split rather than cross-validation. While this approach was sufficient for comparing relative model behaviour and exploring bias–variance effects, cross-validation could provide a more robust and reliable estimate of prediction accuracy for both models. Another improvement that could be made would be evaluating the models on more than just accuracy. Although it was still sufficient to evaluate the models, alternative metrics such as precision or recall may offer further insight, particularly in medical contexts where inaccuracies or error can have real-life negative consequences for many people.

Overall, this project highlights that careful hyper-parameter tuning is essential for both algorithms. The results reinforce the principle that increased complexity does not guarantee improved performance and that identifying the sweet spot in the bias-variance model can lead to significant gains in accuracy.

Conclusions

This project compared k-Nearest Neighbours and Decision Trees on a heart disease classification task, focusing on the effect of model complexity and the bias-variance sweet spot. Both models performed best at low complexity, with accuracy declining as complexity increased, highlighting the importance of hyper-parameter tuning.

The Decision Tree achieved optimal performance at a shallow depth, while kNN performed best with a very small neighbourhood size and achieved higher peak accuracy overall. These results suggest that local similarity between patients is particularly informative for this dataset, though for another dataset, or if cross validation was used, the results could be different. It would be worth while to explore cross-validation, alternative evaluation metrics, or additional datasets to further assess model robustness.

However, it was beneficial to me to be able to get a firm grasp on how these particular models worked, how they can be tuned, and also to have me spend time

thinking about how they are beneficial, and also how they are limited and could potentially be improved.

References

[1] Diagram for bias-variance from course work

[2] UCI Machine Learning Repository (n.d.) *Heart Disease Data Set*. Available at: https://archive.ics.uci.edu/dataset/45/heart+disease (Accessed: 29/12/2025).